Combining Walk-Text and Label Deconvolution Embeddings for SAGN-Based Product Node Classification

Junhao Shen, Rishi Puri, Meghana Shanthappa

Abstract

We evaluate a product-node classification pipeline that combines two text-derived representations: released **Label Deconvolution (LD)** BERT embeddings and **frozen EmbeddingGemma-300m walk-text embeddings (WalkLM-style)**. LD supplies task-oriented features learned with graph-aware supervision. The walk-text branch exposes descriptions of products connected by short graph walks to a pretrained language encoder. We concatenate 768-dimensional LD features, 768-dimensional walk-text features and 100 raw OGB attributes to form a **1,636-dimensional node representation**. [2, 3, 8]

The downstream model uses **SAGN with Self-Label-Enhanced training (SLE)** to combine graph-propagated feature summaries and label information. A validation-selected **Correct & Smooth** stage then refines the predictions. Both language encoders remain fixed during downstream training. The method combines existing components through feature concatenation rather than joint encoder fine-tuning. [4, 5, 8]

We evaluate this external-data pipeline on the official **ogbn-products** split, containing **2,449,029 products and 47 categories**. Across ten independent predictor runs with seeds 10-19, the full pipeline after selected C&S achieves **87.4984% +/- 0.0490% test accuracy** and **94.0584% +/- 0.0433% validation accuracy**, reported as mean +/- sample standard deviation. Its test point estimate is **0.1284 percentage points** above the published GLEM+GIANT+SAGN+SCR score of 87.37%. This is a numerical comparison, not an official OGB ranking or a statistical-significance result. [1, 8, 9]

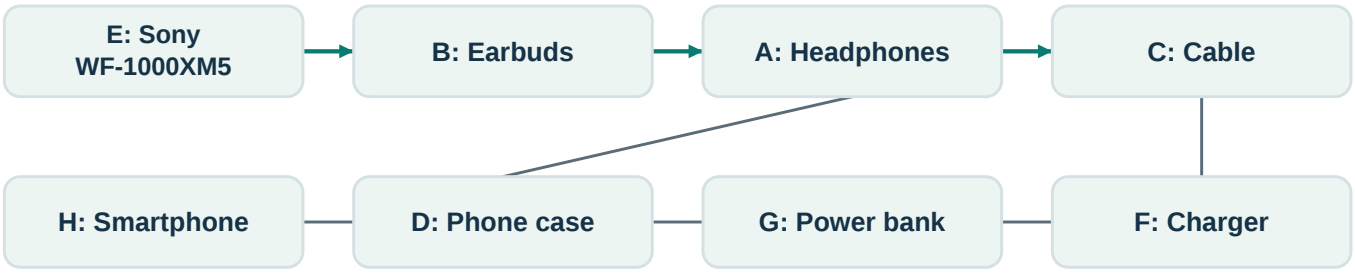
The result describes the complete feature, training and post-processing pipeline. No v2 ablation isolates the contribution of walk-text. Earlier v1 exploration viewed test performance and informed choices inherited by v2; the later validation lock does not erase that history. We provide the worked examples, experimental configuration, per-seed results and artifact-level reproducibility boundaries needed to interpret the reported result. [8]

Product graph and feature pipeline

An illustrative example used throughout the following explanations.

The task is node classification: predict a category for each product.

One product graph throughout



Undirected toy links; teal arrows mark the selected walk E - B - A - C.

All names, descriptions, links and small numerical examples in this report are **illustrative**. The Sony product name is real, but these are not verified OGB items or observed co-purchase links. The actual benchmark has 47 categories; two-category examples only make the arithmetic easier.

Input for each product	Size	What it supplies
Released LD-BERT embedding	768 values	Node text trained with graph-aware label supervision
Frozen walk-text embedding	768 values	A frozen LM jointly reads root and visited-product text
Raw OGB feature vector	100 values	The benchmark's original node attributes
Concatenated input	1,636 values	Keep all three feature groups; do not average them

Why combine LD and walk-text?

Released LD supplies task-oriented, graph-aware text features. A frozen language model encodes walk-text context. Their combination with raw features gives SAGN/SLE a 1,636-dimensional input. The complete pipeline achieves **87.4984% +/- 0.0490% test accuracy** over ten runs. An ablation is needed to isolate walk-text's gain. [2, 3, 8]

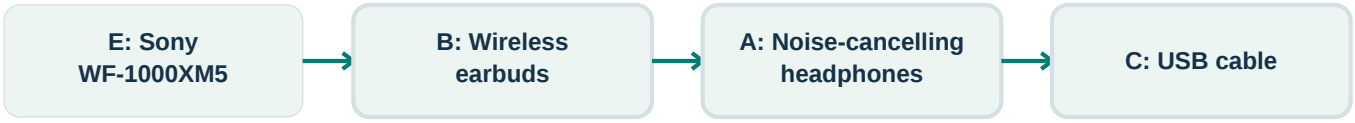
Product text + graph -> LD / walk-text feature vectors -> **SAGN/SLE** -> class predictions -> **Correct & Smooth** -> refined class predictions. [8]

Reading guide

Walk-text: page 3. LD: pages 4-5. SAGN/SLE: page 6. Correct & Smooth: pages 7-8. Training setup: page 9. Results: page 10. Protocol and limitations: page 11. Terms and sources: page 12.

1. Walk-text: paths become vectors

Read the product's text together with text from products reached along graph links.



Step 1 - choose a walk

Start at Sony earbuds E. One possible walk is **E - B - A - C**: Sony earbuds, generic earbuds, headphones, then a cable. This is **four visited nodes and three edge transitions**. Another possible route is E - B - A - D. A random walk can revisit a node.

Step 2 - render the visited attributes as text

Schematic input: "Sony WF-1000XM5 earbuds" | "wireless earbuds" | "noise-cancelling headphones" | "USB cable"

The product name is an attribute; E is just its node identifier. No category labels are inserted into this walk text. The schematic above is for readability: the reconstruction code actually separates node text with the literal word **cites**, even for products. [8]

Step 3 - encode, pool and normalize

Your pipeline uses a frozen **google/embeddinggemma-300m** model. It averages final token states using the attention mask, then L2-normalizes the result. Each product receives one 768-value vector. This context vector stays separate from the LD vector until concatenation. [8]

Tiny pooling example	Value
Three real token states; padding ignored	[1, 0], [0, 2], [2, 1]
Masked mean	[(1+0+2)/3, (0+2+1)/3] = [1, 1]
L2 normalization	[1, 1] / sqrt(2) = [0.7071, 0.7071]

These two coordinates illustrate pooling only. Real token states and embeddings have 768 coordinates, with no fixed coordinate named Audio or Accessories.

Important details of this experiment

Four walks are generated per node, but the **160-token limit retains only the first rendered walk's prefix**; walks 2-4 do not reach the encoder. Up to 160 text characters per visited node are rendered. Thus more generated walks did not mean more encoded context in this artifact. [8]

Original WalkLM fine-tunes a language model on walk-derived text. This project borrows the walk-to-text idea but keeps the encoder frozen: **WalkLM-style, not a reproduction of original WalkLM training**. Freezing reuses existing language knowledge without encoder updates; it does not prove fine-tuning would be worse. [3, 8]

2. LD: train features for graph mixing

LD means Label Deconvolution, not simply Language-Derived.

A graph model later mixes each product's features with neighbor features. LD makes text-encoder supervision account for that future mixing, instead of treating every product as isolated. It learns graph-aware training targets while the stored correct class remains unchanged. Unlike walk-text, LD brings upstream task supervision into the text features; it is not the same encoding step. [2]

A practical-style numerical example

Use two toy classes, ordered **[Audio, Other]**. Suppose these four nodes have known training labels. A has neighbors B, C and D in our graph.

Product	Known class	One-hot label
A: Headphones	Audio	[1, 0]
B: Earbuds	Audio	[1, 0]
C: Cable	Other	[0, 1]
D: Phone case	Other	[0, 1]

A simple neighbor-label mean is ([1,0] + [0,1] + [0,1]) / 3 = **[1/3, 2/3]**. For illustration, choose 70% own-label information and 30% one-hop label information.

Adjusted training target for A
0.7 x [1, 0] + 0.3 x [1/3, 2/3] = **[0.8, 0.2]**

Read the two entries as supervision scores for Audio and Other. They are **not** a new stored label, an observed prediction, or the 768-dimensional embedding. A's correct class is still Audio; the final prediction is evaluated against that original class.

What is learned, and what was reused?

LD training in the method	Your experiment
Combine multi-hop label signals with learned weights; train the text encoder using adjusted supervision.	Reuse the released 768-dimensional LD-BERT features. This report does not claim you retrained LD.
Extract node features, then train a downstream graph model with the encoder fixed.	Concatenate LD features with walk-text and raw features, then train SAGN/SLE.

The 70/30 weights and simple mean are teaching choices, not recovered LD settings. Practical LD uses learned multi-hop label combinations and normalizes targets for classification. Its exact graph normalization need not equal this neighbor mean. [2]

3. Why show [1.5, -0.5]?

A theoretical inverse-mixing illustration, not the practical LD targets or actual released embeddings.

Keep only two connected nodes: **A headphones** and **D phone case**; ignore their other links for this example. Order scores as **[Audio, Accessories]**. Let a toy graph operator mix 75% own scores and 25% neighbor scores.

Scores before graph mixing	Headphones A	Phone case D
Original one-hot values	[1, 0]	[0, 1]
Inverse-adjusted toy scores	[1.5, -0.5]	[-0.5, 1.5]

Using the original values

A: 0.75 x [1, 0] + 0.25 x [0, 1] = **[0.75, 0.25]** -> Audio
D: 0.75 x [0, 1] + 0.25 x [1, 0] = **[0.25, 0.75]** -> Accessories

Using the inverse-adjusted values

A's Audio score: 0.75 x 1.5 + 0.25 x (-0.5) = **1**
A's Accessories score: 0.75 x (-0.5) + 0.25 x 1.5 = **0**
A becomes **[1, 0]**; the symmetric calculation gives D = **[0, 1]**.

The larger positive score and the negative score compensate for later neighbor mixing. This makes LD's inverse-mapping motivation tangible. Both original and adjusted cases already predict A as Audio; the example demonstrates **exact recovery of toy target scores**, not a measured accuracy gain. [2]

4. SAGN/SLE: use the right distance

SAGN = Scalable and Adaptive Graph Neural Networks.

Toy fusion for E: LD [2.0, 0.5] || walk-text [0.6, 0.8] || raw [0.3]
= **[2.0, 0.5, 0.6, 0.8, 0.3]**. Real input: **768 + 768 + 100 = 1,636**.

Toy latent coordinates, not class probabilities; feature blocks are joined, not averaged or semantically aligned.

SAGN precomputes 0-, 1- and 2-hop summaries from the **combined feature vectors**. Each has its own small MLP, followed by **node-specific attention across hops**. A residual and classifier produce class scores. For E, B is 1 hop away and A is reached in 2 hops. [4, 8]

Same eight products: illustrative attention weights

Product	0-hop	1-hop	2-hop
A: Headphones	80%	15%	5%
B: Earbuds	70%	25%	5%
C: Cable	40%	40%	20%
D: Phone case	60%	30%	10%
E: Sony WF-1000XM5	20%	60%	20%
F: Charger	65%	25%	10%
G: Power bank	55%	35%	10%
H: Smartphone	75%	20%	5%

Every row sums to 100%. All values are invented for explanation, not measured attention. These weights mix **encoded hop summaries**, not individual neighbors or class probabilities. Summaries can include own-node information and revisits, not only nodes exactly that distance away.

A numerical mixture for Sony earbuds E

Suppose its three encoded summaries are h0 = [1,0], h1 = [0.6,0.4], h2 = [0.8,0.2]. These are two toy feature coordinates, not Audio/Other probabilities.

0.2 x h0 + 0.6 x h1 + 0.2 x h2 = **[0.72, 0.28]**
The residual and classifier come next; [0.72, 0.28] is not yet the final class prediction.

What SLE adds

Self-Label-Enhanced training combines label propagation with staged self-training. Confident predictions become pseudo-labels for later stages; they are model guesses, not revealed test answers. Your run used four stages. SLE is a training framework around the predictor, not another name for SAGN. [4, 8]

5. Correct: spread known errors

Correct & Smooth refines class predictions after the model. It does not retrain either text encoder. [5]

Teaching subgraph: A-B, A-C, A-D, B-E only. Ignore F/G/H and their links. Use neighbor means without self-loops, alpha = 0.5, scale = 1, and two iterations per stage. These are not the actual experiment settings.

Start with predictions and only two known labels

Product	Known training label	Base [Audio, Other]
A: Headphones	Audio: [1,0]	[0.70, 0.30]
B: Earbuds	Unknown	[0.60, 0.40]
C: Cable	Unknown	[0.20, 0.80]
D: Phone case	Other: [0,1]	[0.10, 0.90]
E: Sony WF-1000XM5	Unknown	[0.40, 0.60]

Only A and D provide true labels. Their residuals are **label - prediction**:
A: [1,0] - [0.70,0.30] = **[+0.30,-0.30]**
D: [0,1] - [0.10,0.90] = **[-0.10,+0.10]**.

Initialize B/C/E residuals at zero because their labels are unknown, not because their predictions are perfect. Propagate these errors, then add the result to each original prediction. [5, 6]

At each pass: **new = 0.5 x mean(neighbors' current residuals) + 0.5 x fixed residual seed**. After each correction pass, restore A and D to their known residual seeds. [6, 7]

Audio residual values

Node	Seed	Pass 1	Pass 2	Corrected Audio
A: Headphones	0.3000	0.3000	0.3000	1.0000
B: Earbuds	0.0000	0.0750	0.0750	0.6750
C: Cable	0.0000	0.1500	0.1500	0.3500
D: Phone case	-0.1000	-0.1000	-0.1000	0.0000
E: Sony WF-1000XM5	0.0000	0.0000	0.0375	0.4375

For B: 0.5 x mean(A: 0.30, E: 0) = **0.075**. On the next pass, E receives 0.5 x 0.075 = **0.0375**. Its corrected Audio score becomes 0.40 + 0.0375 = **0.4375**. The Other residual is the negative of the Audio residual in this two-class example.

6. Smooth: spread prediction scores

Correct spreads residuals; Smooth spreads label/prediction scores.

Start from the corrected predictions. Replace A and D with their known one-hot training labels once before smoothing. In this fixed-scale teaching variant, they are **not clamped after each smoothing pass**. [6]

new = 0.5 x mean(neighbors' current scores) + 0.5 x fixed smoothing seed
The restart term always uses the initial smoothing seed, not the previous iteration. [7]

Follow Sony earbuds E

E has only neighbor B in this five-node subgraph. E's fixed seed is 0.4375 and B's seed is 0.675.
Pass 1: 0.5 x 0.675 + 0.5 x 0.4375 = **0.55625**.
After pass 1, B is 0.696875.
Pass 2: 0.5 x 0.696875 + 0.5 x 0.4375 = **0.5671875**.

Node	Fixed seed	Smooth pass 1	Smooth pass 2
A: Headphones	1.000000	0.670833	0.811979
B: Earbuds	0.675000	0.696875	0.644271
C: Cable	0.350000	0.675000	0.510417
D: Phone case	0.000000	0.500000	0.335417
E: Sony WF-1000XM5	0.437500	0.556250	0.567188

Before and after the complete toy procedure

Product	Base	After Correct	After Smooth
A: Headphones	[0.700, 0.300]	[1.000, 0.000]	[0.812, 0.188]
B: Earbuds	[0.600, 0.400]	[0.675, 0.325]	[0.644, 0.356]
C: Cable	[0.200, 0.800]	[0.350, 0.650]	[0.510, 0.490]
D: Phone case	[0.100, 0.900]	[0.000, 1.000]	[0.335, 0.665]
E: Sony WF-1000XM5	[0.400, 0.600]	[0.438, 0.562]	[0.567, 0.433]

E switches from Other to Audio, but its true class was never supplied. That change alone is **not evidence of better accuracy**. Cable C also moves slightly toward Audio; that could be wrong if its true class were Other. Cross-category links can make smoothing harmful.

Keep the stages separate. LD changes encoder supervision during upstream training. SAGN learns feature/hop combinations. C&S changes prediction scores afterward. Only training labels may seed this example.

7. Experimental setup

Official OGB split, fixed input features and ten independent downstream training runs.

Dataset property	Recorded value
Dataset / version / metric	ogbn-products / OGB 1.3.6 / node accuracy
Nodes / categories	2,449,029 / 47
Graph size	61,859,140 undirected links; 123,718,280 directional records
Train / validation / test nodes	196,615 / 39,323 / 2,213,091
Split / external data	Official sales-rank split / Yes

7.1 Feature extraction and predictor configuration

Setting	Actual choice
LD branch	Released bert-base-uncased hidden states, 768 dimensions
Walk-text encoder	Frozen google/embeddinggemma-300m; 768 dimensions
Walk construction	Seed 42; 4 generated walks; 4 visited nodes per walk
Text limits / pooling	160 characters per node; 160 tokens total; masked mean + L2
Effective context	Only the first rendered walk's prefix reaches the encoder
Feature hops / label hops	2 / 9
Hidden channels / attention heads	512 / 1
Learning rate / weight decay	0.001 / 0
Train / evaluation batch sizes	50,000 / 100,000
Dropout: model / attention / input / label	0.5 / 0.4 / 0.2 / 0.5
Stage epochs / pseudo-label thresholds	300, 100, 100, 100 / 0.95, 0.95, 0.999
Predictor seeds / aggregation chunks	10 through 19 / 4

7.2 Selection and label boundary

Training labels seed SAGN/SLE label features and C&S. Validation labels support model selection and early stopping. V2 masks true test labels before training/preprocessing and suppresses development-time test metrics. Released LD features retain their upstream provenance. [8]

The 27-candidate C&S grid selects the largest mean validation accuracy, then lower sample SD, then filename for ties. Its winner uses 50/50 propagation steps, alpha 0.35/0.60, DAD/DAD and scale 20. Candidate records and the selected lock precede the guarded final evaluation. [8]

8. Results and baseline comparison

Mean +/- unbiased sample SD across seeds 10-19; not a prediction ensemble.

Evaluation	Mean accuracy	Sample SD
Validation before C&S	93.9204%	0.0522%
Selected validation after C&S	94.0584%	0.0433%
Final test after C&S	87.4984%	0.0490%

Selected C&S improves validation accuracy by **0.1381 percentage points**. V2 did not score pre-C&S test predictions, so this does not measure a test-set C&S gain. [8]

8.1 Per-seed accuracy

Seed	Validation before C&S	Validation after C&S	Final test
10	93.8865%	94.0645%	87.5155%
11	93.9704%	94.0544%	87.4735%
12	93.8077%	93.9526%	87.5673%
13	93.9577%	94.0722%	87.5404%
14	93.9704%	94.0747%	87.4646%
15	93.9654%	94.1078%	87.4907%
16	93.9069%	94.0467%	87.4027%
17	93.8789%	94.0722%	87.5148%
18	93.9196%	94.1027%	87.4660%
19	93.9399%	94.0366%	87.5488%

8.2 Published external-data comparator scores

Method	Test accuracy	Our numerical margin
GLEM+GIANT+SAGN+SCR	87.37% +/- 0.06%	+0.1284 pp
LD+GIANT+SAGN+SCR	87.18% +/- 0.04%	+0.3184 pp
Our complete pipeline	87.4984% +/- 0.0490%	Not officially ranked

Published rows checked 2 November 2026. These compare different pipelines, resource budgets and historical protocols; they are not a matched ablation or a significance result. OGB determines official ranking. [9]

The mean uses n=10; sample SD uses n-1=9. This SD measures seed variability conditional on fixed graph, split and features. It excludes encoder/corpus uncertainty and method-selection bias. Paired baseline reruns are needed for stronger statistical conclusions.

9. Evidence, resources and limitations

Artifact verification supports the reported full-stack result, with explicit boundaries.

9.1 Evaluation history

V1 inspected test performance during encoder, feature, SAGN and C&S exploration. V2 inherited those choices, then fixed seeds 10-19 and a validation-only selection procedure. The committed lock creates a forward selection boundary; it does not undo prior test exposure. [8]

The preserved history contains **two post-lock v2 test computations**: the original guarded evaluation and a later reproduction using released predictions with a separate access marker. Recomputing released predictions is not fresh end-to-end retraining. This revision only recomputes summary arithmetic, without new training or benchmark tests. [8]

9.2 Reproducibility and resources

Item	Evidence / scope
Fixed feature alignment	N x768 LD, N x768 walk-text, N x100 raw, in OGB node order
Walk-text reconstruction	128-node prefix exactly matches 98,304 archived float32 values; not a full regeneration proof
Final predictor parameters	5,010,014, from recorded runtime logs
Conservative multi-stage count	423,228,606, including upstream encoders and LD head
Float32 fused feature matrix	16,026,445,776 bytes (14.93 GiB); not peak RAM/GPU usage
Original generation environment	Original modified source, software versions, compute dtype, GPU and resolved model revision were not captured

The reconstruction pins EmbeddingGemma revision 57c266a7 and documents source revisions and tensor hashes. A matched prefix is evidence of local agreement, not complete regeneration. No measured runtime or memory-efficiency advantage is established. [8]

9.3 Studies needed to isolate the combination

Prospective control	What it would isolate
LD+raw versus LD+walk-text+raw	Incremental feature-block value with matched seeds and capacity controls
Own text versus real/shuffled walk text	Neighbor context versus text volume and spurious context
SAGN, +SLE, +C&S across feature variants	Representation, self-training and post-processing contributions

These are proposed studies, not completed results. Select them using training/validation only and seek a genuinely untouched evaluation setting. The measured full-pipeline score is supported; an isolated walk-text gain, a novel fusion algorithm and statistical superiority remain unestablished.

10. Quick reference and sources

The same word can mean different things at different stages.

Term	Meaning in this report
Node ID / attribute	E identifies a node; Sony WF-1000XM5 is product text attached to it.
One-hot / one-hop	[1,0] encodes a class; one hop is one graph edge.
Embedding	A latent feature vector, not automatically class probabilities.
Feature concatenation	Join LD, walk-text and raw coordinates end to end: 768 + 768 + 100. No averaging or joint encoder training is implied.
Label / prediction	The known answer versus the model's estimated class scores.
LD target / SAGN weight	A training-supervision vector versus attention across feature hops.

GCN, GAT and SAGN in one comparison

Model	Where the aggregation weights act
GCN	Degree-normalized neighbor messages during graph layers.
GAT	Learned attention between individual neighbors.
SAGN	Learned attention across precomputed, encoded hop summaries.

Primary references and project evidence

[1] OGB: ogbn-products dataset and task.

[2] Shi et al. Label Deconvolution paper and author implementation / released features.

[3] Tan et al. WalkLM, NeurIPS 2023.

[4] Sun, Gu and Hu. SAGN with Self-Label-Enhanced training.

[5] Huang et al. Correct & Smooth paper, ICLR 2021.

[6] PyG source: CorrectAndSmooth (fixed-scale and smoothing behavior).

[7] PyG source: LabelPropagation (fixed restart seed).

[8] Project snapshot 3b793c8: README, provenance, protocol, validation lock, selected candidate, final test JSON.

[9] Official OGB products leaderboard, checked 2 November 2026.

The central combination: released LD features supply a task-oriented view; frozen WalkLM-style features supply a walk-context view. We concatenate both with raw features, then train SAGN/SLE and apply Correct & Smooth. The score belongs to the full pipeline, not an isolated measured walk-text gain.